

Rajarata University of Sri Lanka
Department of Computing



ICT1402

PRINCIPLES OF PROGRAM DESIGN & PROGRAMMING

Lecture 10

Program Control Structures – Repetition and Loops

K.A.S.H. Kulathilake

Ph.D., M. Phil., MCS, B.Sc. (Hons) IT, SEDA(UK)

Objectives

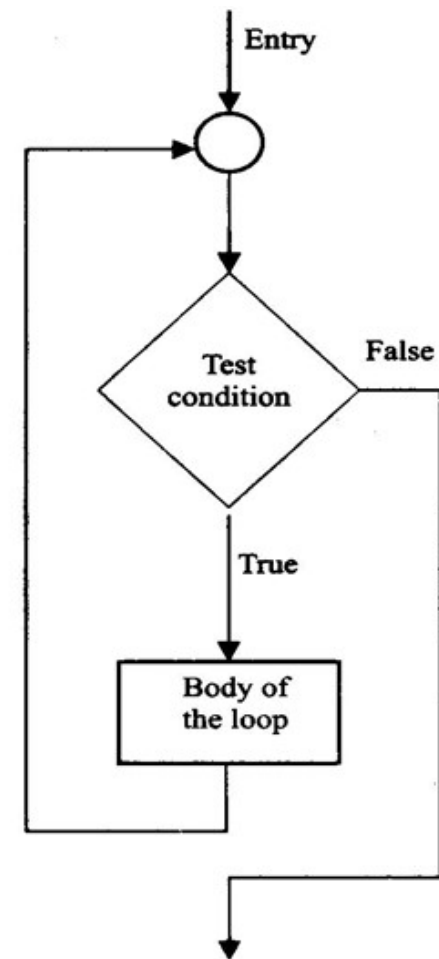
- At the end of this lecture students should be able to;
 - Describe the looping structures in C programming language.
 - Practice the control flow of different looping structures in C programming language.
 - Practice the variants in control flow of different looping structures in C programming language.
 - Apply taught concepts for writing programs.

Introduction

- One of the fundamental properties of a computer is its ability to repetitively execute a set of statements.
- These *looping* capabilities enable you to develop concise programs containing repetitive processes that could otherwise require thousands or even millions of program statements to perform.
- If a loop continue forever, it is called infinite loop.
- A Looping consist of 2 segments.
 - Body of the Loop
 - Control statement
- A control structure may be classified either,
 - As the entry controlled loop or
 - Exit-controlled loop

Introduction (Cont...)

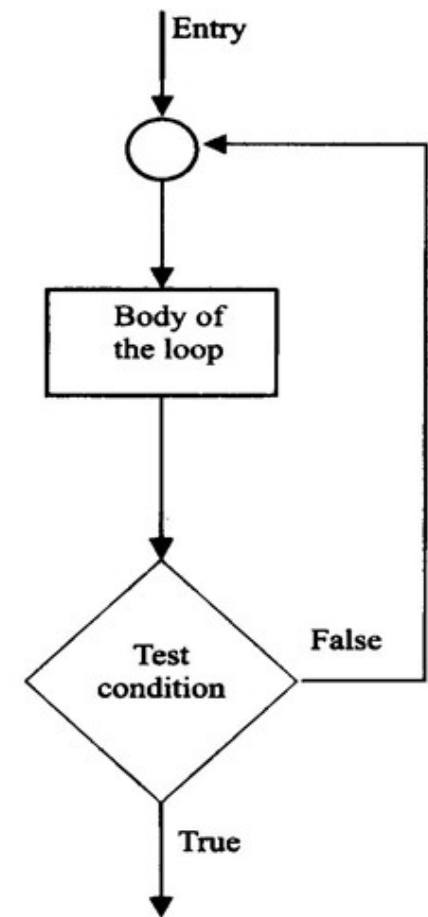
- Entry controlled loop
 - In entry controlled loop, the control conditions are tested before the start of loop execution.
 - If the conditions are not satisfied, then the body of the loop will not be executed.



(A) Entry control

Introduction (Cont...)

- Exit-controlled loop
 - The test is performed at the end of the body of the loop and therefore the body is executed unconditionally for the first time.



(b) Exit control

Introduction (Cont...)

- There are five parts of a looping construct;
 - Loop control variable (Declaration)
 - Initial value of the loop control variable (Initialization)
 - Loop control condition (Test condition)
 - Repetitive statements (Body of the Loop)
 - Increment the loop control variable (Increment)

Introduction (Cont...)

- The C programming language contains three different program statements for program looping.
 - For statement,
 - While statement,
 - Do statement.

The while Statement

- The while is an entry-controlled loop statement.
- The basic format of the while statement is,

```
initialization;  
while (test condition)  
{  
    body of the loop  
}
```


The while Statement (Cont...)

```
#include <stdio.h>
```

```
int main ()  
{
```

Loop variable declaration and
initialization

```
    int x =1;
```

```
    while (x <= 10)
```

test condition

```
    {
```

```
        printf ("%i ", x);
```

```
        x =x+1;
```

Loop body

```
    }
```

```
    return 0;
```

Increment

```
}
```

The while Statement (Cont...)

- The *test_condition* specified inside the parentheses is evaluated.
- If the result of the expression evaluation is TRUE, the *body of the loop* that immediately follows is executed.
- After execution of this statement (or statements if enclosed in braces), the *test_condition* is once again evaluated.
- If the result of the evaluation is TRUE, the *body of the loop* is once again executed.
- This process continues until the *test_condition* finally evaluates as FALSE, at which point the loop is terminated.
- Execution of the program then continues with the statement that follows the *body of the loop*.

The while Statement (Cont...)

```
#include <stdio.h>
int main (void)
{
    int count = 1;
    while ( count <= 5 )
    {
        printf ("%i\n", count);
        ++count;
    }
    return 0;
}
```

The while Statement (Cont...)

- The next program computes the *greatest common divisor* of two integer values.
- The greatest common divisor (*gcd*) of two integers is the largest integer value that evenly divides the two integers.

The while Statement (Cont...)

```
#include <stdio.h>
int main (void)
{
    int u, v, temp;
    printf ("Please type in two nonnegative integers.\n");
    scanf ("%i%i", &u, &v);
    while ( v != 0 )
    {
        temp = u % v;
        u = v;
        v = temp;
    }
    printf ("Their greatest common divisor is %i\n", u);
    return 0;
}
```

The while Statement (Cont...)

```
#include <stdio.h>
int main (void)
{
    int number, right_digit;
    printf ("Enter your number.\n");
    scanf ("%i", &number);
    while ( number != 0 )
    {
        right_digit = number % 10;
        printf ("%i", right_digit);
        number = number / 10;
    }
    printf ("\n");
    return 0;
}
```

The do- while Statement

- A do loop is much like a while loop, except that the expression is tested at the bottom of the loop, rather than at the top.

```
initialization;  
do  
{  
    Body of the loop  
}  
while (test condition);
```

The do- while Statement (Cont...)

```
#include <stdio.h>
```

```
int main ()  
{
```

```
    int x =1;
```

```
    do
```

```
    {
```

```
        printf ("%i , ", x);
```

```
        x =x+1;
```

```
    }
```

```
    while (x <= 10);
```

```
    return 0;
```

```
}
```

Loop variable declaration and
initialization

Loop body

Increment

test condition

The do- while Statement (Cont...)

- Execution of the do statement proceeds as follows:
 - The *body of the loop* is executed first.
 - Next, the *test_condition* inside the parentheses is evaluated.
 - If the result of evaluating the *test_condition* is TRUE, the loop continues and the *program statement* is once again executed.
 - As long as evaluation of the *test_condition* continues to be TRUE, the *body of the loop* is repeatedly executed.
 - When evaluation of the expression proves FALSE, the loop is terminated, and the next statement in the program is executed in the normal sequential manner.

The do- while Statement (Cont...)

```
#include <stdio.h>
int main ()
{
    int number, right_digit;
    printf ("Enter your number.\n");
    scanf ("%i", &number);
    do
    {
        right_digit = number % 10;
        printf ("%i", right_digit);
        number = number / 10;
    }
    while ( number != 0 );
    printf ("\n");
    return 0;
}
```

The For Statement

- The for loop is an entry-controlled loop that provides a more concise loop control structure.
- The general form of the for loop is,

```
declaration
for (initialization ; test condition; increment)
{
    Body of the loop
}
```

The For Statement (Cont...)

```
#include <stdio.h>
```

```
int main ()  
{
```

```
    int x;
```

```
    for (x = 0; x < 10; x = x + 1 )
```

```
        printf ("%i ", x);
```

```
    return 0;
```

```
}
```

Loop variable

Initialization, test condition, increment

x++ is also possible

Braces are not mandatory to state the scope/ boundary of the loop body if it has single statement to be executed.

The For Statement (Cont...)

- The **for** statement allows for negative increments.

```
#include <stdio.h>
int main ()
{
    int x;
    for (x = 10; x>0; x-- )
        printf ("%i\n", x);
    return 0;
}
```

The For Statement (Cont...)

- In summary, execution of the for statement proceeds as follows:
 1. The initialization expression is evaluated first. This expression usually sets a variable that will be used inside the loop, generally referred to as an loop variable and it has an initial value.
 2. The looping condition is evaluated. If the condition is not satisfied (the expression is FALSE), the loop is immediately terminated. Otherwise, execution continues with the program statement that immediately follows the loop.
 3. The program statement that constitutes the body of the loop is executed.
 4. The looping expression is evaluated. This expression is generally used to change the value of the index variable, frequently by adding 1 to it or subtracting 1 from it.
 5. Return to step 2.

The For Statement (Cont...)

- Triangular Number Generator

```
#include <stdio.h>
int main (void)
{
    int n, triangularNumber;
    printf ("TABLE OF TRIANGULAR NUMBERS\n\n");
    printf (" n Sum from 1 to n\n");
    printf ("--- -----\n");
    triangularNumber = 0;
    for ( n = 1; n <= 10; ++n ) {
        triangularNumber += n;
        printf (" %2i %i\n", n, triangularNumber);
    }
    return 0;
}
```

The For Statement (Cont...)

```
#include <stdio.h>
int main (void)
{
    int n, number, triangularNumber;
    printf ("What triangular number do you want? ");
    scanf ("%i", &number);
    triangularNumber = 0;
    for ( n = 1; n <= number; ++n )
        triangularNumber += n;
    printf ("Triangular number %i is %i\n", number,
           triangularNumber);
    return 0;
}
```


The For Statement (Cont...)

```
#include <stdio.h>
int main (void)
{
    int n, number, triangularNumber, counter;
    for ( counter = 1; counter <= 5; ++counter ) {
        printf ("What triangular number do you want? ");
        scanf ("%i", &number);
        triangularNumber = 0;
        for ( n = 1; n <= number; ++n )
            triangularNumber += n;
        printf ("Triangular number %i is %i\n\n", number,
            triangularNumber);
    }
    return 0;
}
```

for Loop Variants

- **Multiple Expressions**

- You can include multiple expressions in any of the fields of the for loop, provided that you separate such expressions by commas.
- For example, in the for statement that begins

```
for ( i = 0, j = 0; i < 10; ++i )
```

...
- The two expressions `i = 0` and `j = 0` are separated from each other by a comma, and both expressions are considered part of the *init_expression* or *variable initialization* field of the loop.

for Loop Variants (Cont...)

- As another example, the for loop that starts

```
for ( i = 0, j = 100; i < 10; ++i, j = j - 10 )
```

...

- sets up two index variables, *i* and *j*; the former initialized to 0 and the latter to 100 before the loop begins.
- Each time after the body of the loop is executed, the value of *i* is incremented by 1, whereas the value of *j* is decremented by 10.

for Loop Variants (Cont...)

```
#include <stdio.h>
int main ()
{
    int x, y ;
    for (x = 1, y = 100; x<=10; x++, y=y-10 )
        printf ("%i ,",x* y);
    return 0;
}
```

for Loop Variants (Cont...)

```
#include <stdio.h>
```

```
int main ()
```

```
{
```

```
    int x, y ;
```

```
    for (x = 1, y = 100; x<=10 && y>=50; x++, y=y-10 )
```

```
        printf ("%i ,", x* y);
```

```
    return 0;
```

```
}
```

for Loop Variants (Cont...)

- Omitting Fields

- This can be done simply by omitting the desired field and marking its place with a semicolon.

```
for ( ; j != 100; ++j )
```

```
...
```

- Or

```
for (j = 0 ; j != 100; ++j )
```

```
...
```

- Or

```
for ( ; j != 100; )
```

```
...
```

for Loop Variants (Cont...)

```
#include <stdio.h>
int main ()
{
    int x =1;
    for (x ; x<=10; x++)
        printf ("%i ", x);
    return 0;
}
```

for Loop Variants (Cont...)

```
#include <stdio.h>
int main ()
{
    int x;
    for (x =1 ; x<=10;) {
        printf ("%i ", x) ;
        x++;
    }
    return 0;
}
```


for Loop Variants (Cont...)

```
#include <stdio.h>
int main ()
{
    int x =1;
    for ( ; x<=10; ) {
        printf ("%i ", x) ;
        x++;
    }
    return 0;
}
```

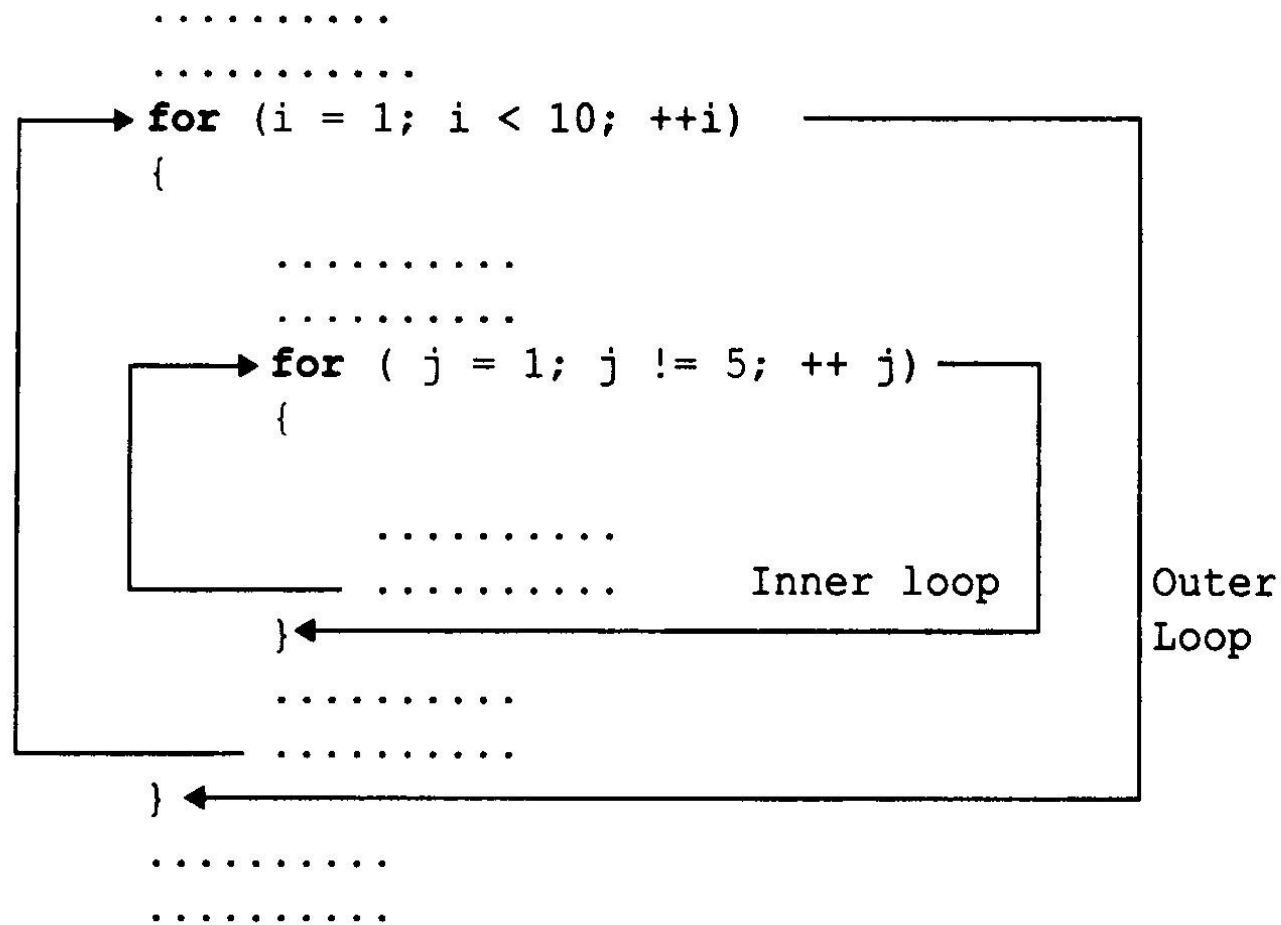
for Loop Variants (Cont...)

- Infinite loop
 - A loop becomes an infinite loop if a condition never becomes false.
 - The **for** loop is traditionally used for this purpose.
 - Since none of the three expressions that form the 'for' loop are required, you can make an endless loop by leaving the conditional expression empty.

for Loop Variants (Cont...)

```
#include <stdio.h>
int main ()
{
    for( ; ; )
    {
        printf("This loop will run forever.\n");
    }
    return 0;
}
```

Nesting of for Loop



Nesting of for Loop (Cont...)

- A program segment to print a multiplication table.

```
.....  
for (row = 1; row <= ROWMAX; ++row)  
{  
    for (column = 1; column <= COLMAX; ++column)  
    {  
        y = row * column;  
        printf ("%4i ", y) ;  
    }  
    printf ("\n") ;  
}
```

- ```
.....
```
- The outer loop controls the rows while the inner one controls the columns.

# Nesting of for Loop (Cont...)

```
#include <stdio.h>
int main ()
{
 const int ROWMAX =12;
 const int COLMAX = 12;
 int row, column, y;

 for (row = 1; row <= ROWMAX; ++row)
 for (column = 1; column <= COLMAX; ++column)
 {
 y = row * column;
 printf ("%4i ", y) ;
 }
 printf ("\n");
 return 0;
}
```

Pay your attention for  
scope declaration.

# When to use for Loop

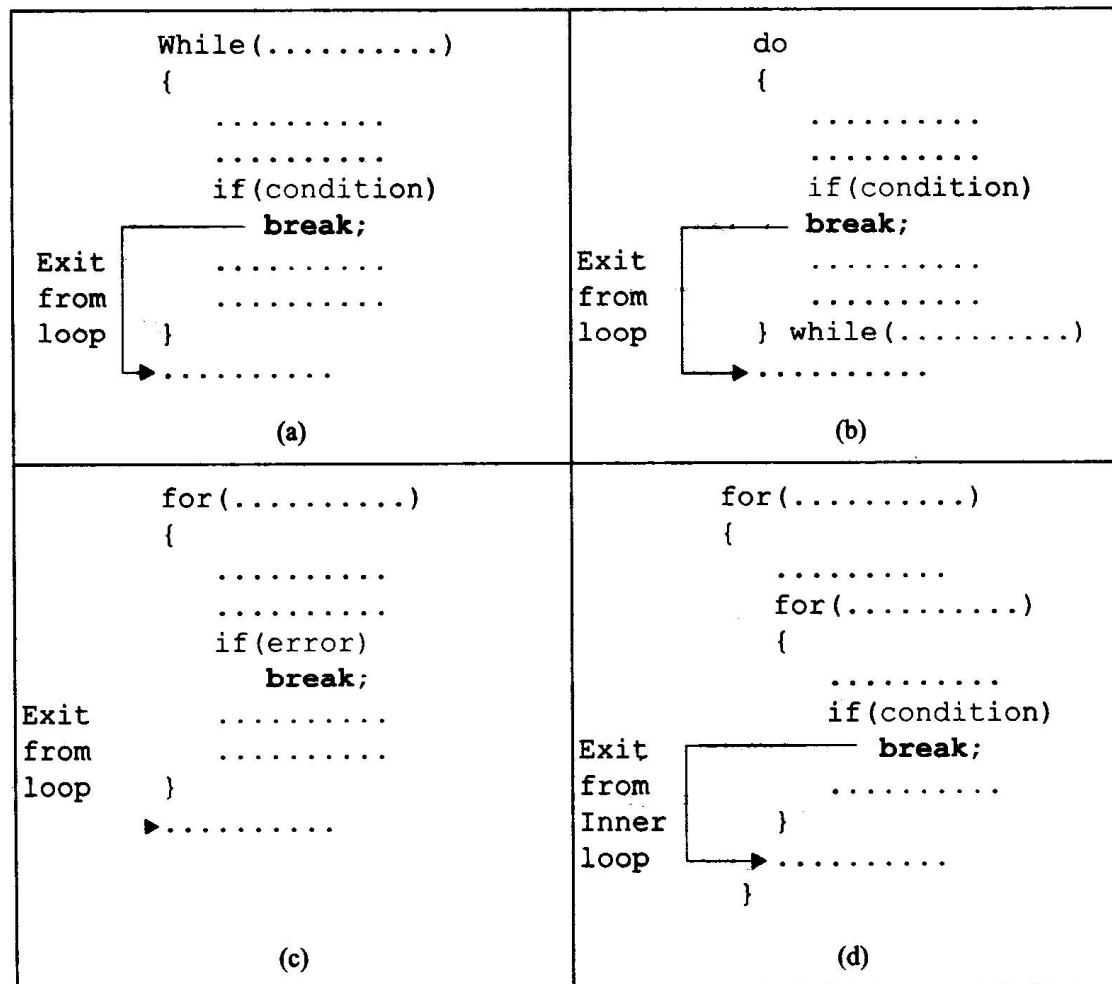
- In general, a loop executed a predetermined number of times is a prime candidate for implementation as a for statement.
- Also, if the initial expression, looping expression, and looping condition all involve the same variable, the for statement is probably the right choice.

# The break Statement

- The break Statement Sometimes when executing a loop, it becomes desirable to leave the loop as soon as a certain condition occurs (for instance, you detect an error condition, or you reach the end of your data prematurely).
- The break statement can be used for this purpose.
- Execution of the break statement causes the program to immediately exit from the loop it is executing, whether it's a for, while, or do loop.
- Subsequent statements in the loop are skipped, and execution of the loop is terminated.
- Execution continues with whatever statement follows the loop.
- If a break is executed from within a set of nested loops, only the innermost loop in which the break is executed is terminated.



# The break Statement (Cont...)



# The break Statement (Cont...)

```
#include <stdio.h>

int main ()
{
 int x = 1;
 while (1)
 {
 printf ("%i ", x);
 if (x == 5)
 break;
 x =x+1;
 }
 return 0;
}
```

# The break Statement (Cont...)

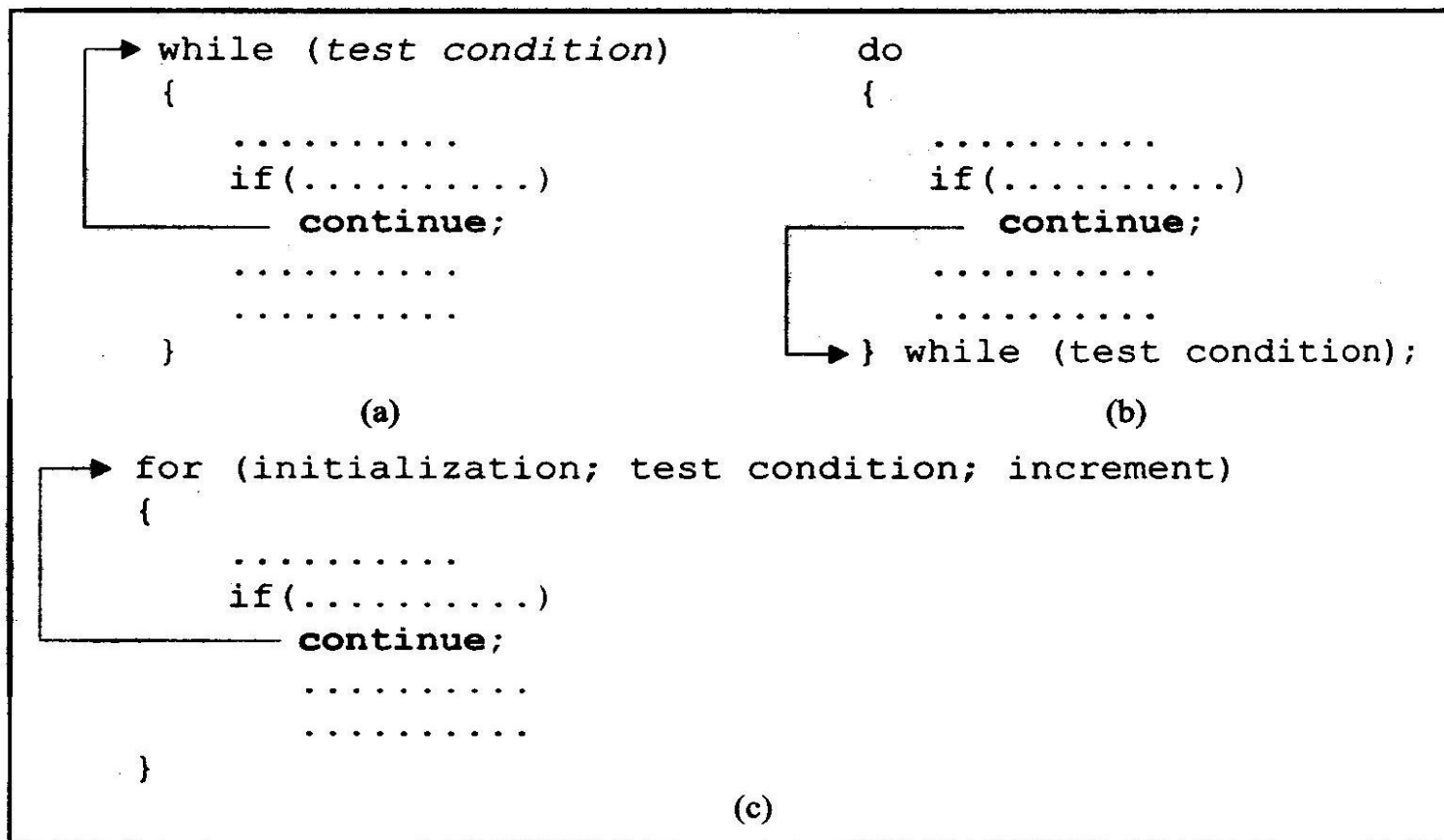
```
#include <stdio.h>

int main ()
{
 int x, y;
 for (x = 1; x<10; x++)
 {
 for (y = 1; y<10; y++)
 {
 printf ("%2i ", x);
 if (y == x)
 break;
 }
 printf ("\n");
 }
 return 0;
}
```

# The continue Statement

- The continue statement is similar to the break statement except it doesn't cause the loop to terminate.
- Rather, as its name implies, this statement causes the loop in which it is executed to be continued.
- At the point that the continue statement is executed, any statements in the loop that appear *after* the continue statement are automatically skipped.
- Execution of the loop otherwise continues as normal.
- The continue statement is most often used to bypass a group of statements inside a loop based upon some condition, but to otherwise continue execution of the loop.

# The continue Statement (Cont...)



# The continue Statement (Cont...)

```
#include <stdio.h>

int main ()
{
 int x, y;
 for (x = 1; x<10; x++)
 {
 for (y = 1; y<10; y++)
 {
 if (y == x)
 continue;
 printf ("%2i ", x);
 }
 printf ("\n");
 }
 return 0;
}
```

# The goto Statement

- A **goto** statement in C programming provides an unconditional jump from the 'goto' to a labeled statement in the same function.
- **NOTE** – Use of **goto** statement is highly discouraged in any programming language because it makes difficult to trace the control flow of a program, making the program hard to understand and hard to modify.
- Any program that uses a goto can be rewritten to avoid them.

# The goto Statement (Cont...)

```
#include <stdio.h>

int main ()
{
 int a = 10;
 LOOP:do {
 if(a == 15) {
 /* skip the iteration */
 a = a + 1;
 goto LOOP;
 }
 printf("value of a: %d\n", a);
 a++;
 }while(a < 20);
 return 0;
}
```



# Objective Re-cap

- Now you should be able to:
  - Describe the looping structures in C programming language.
  - Practice the control flow of different looping structures in C programming language.
  - Practice the variants in control flow of different looping structures in C programming language.
  - Apply taught concepts for writing programs.

# References

- Chapter 05, - Programming in C, 3<sup>rd</sup> Edition, Stephen G. Kochan

Q & A

Next: Arrays